`cache_ext`: Customizing the Page Cache with eBPF

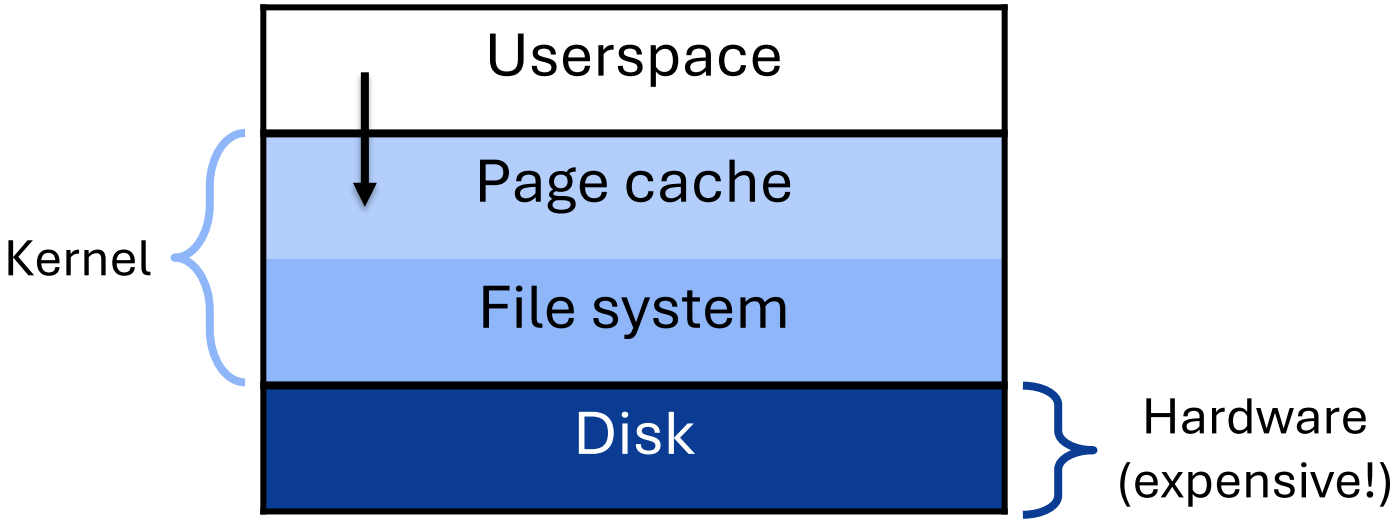
Tal Zussman Columbia
Ioannis Zarkadas Columbia
Jeremy Carin Columbia
Andrew Cheng Columbia
Hubertus Franke IBM
Jonas Pfefferle IBM
Asaf Cidon Columbia

Columbia University
IBM Research

TLDR;

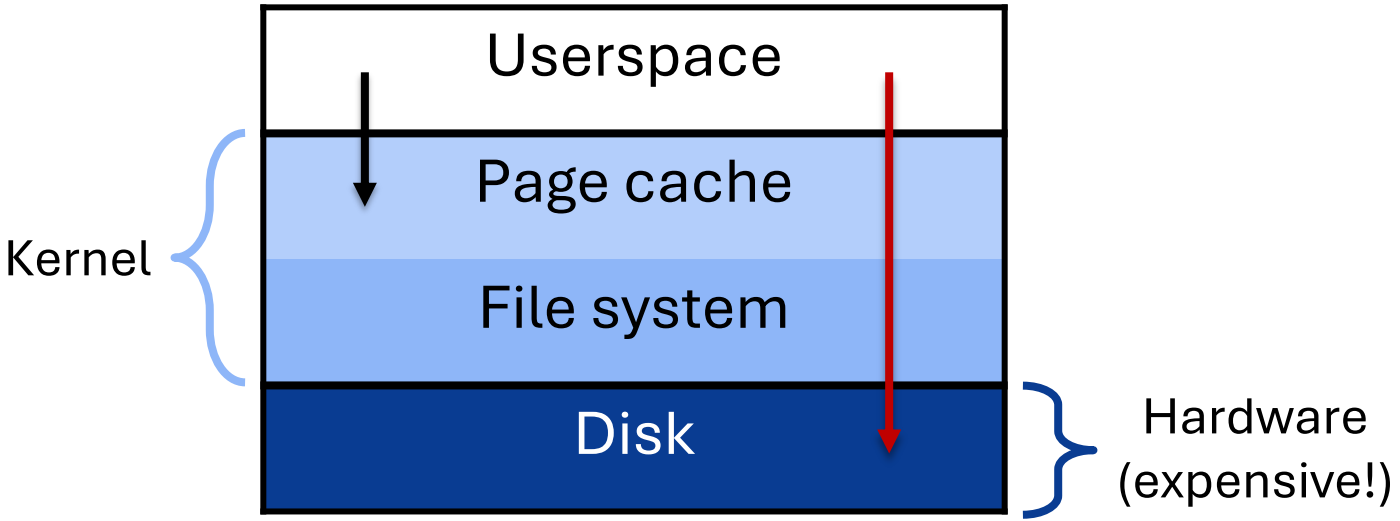
- The page cache doesn’t work so well with some workloads…
- cache_ext allows applications to customize page cache policies to improve performance using eBPF
- Experimentation with policies and application hints are necessary to maximize performance

Linux page cache significantly affects performance



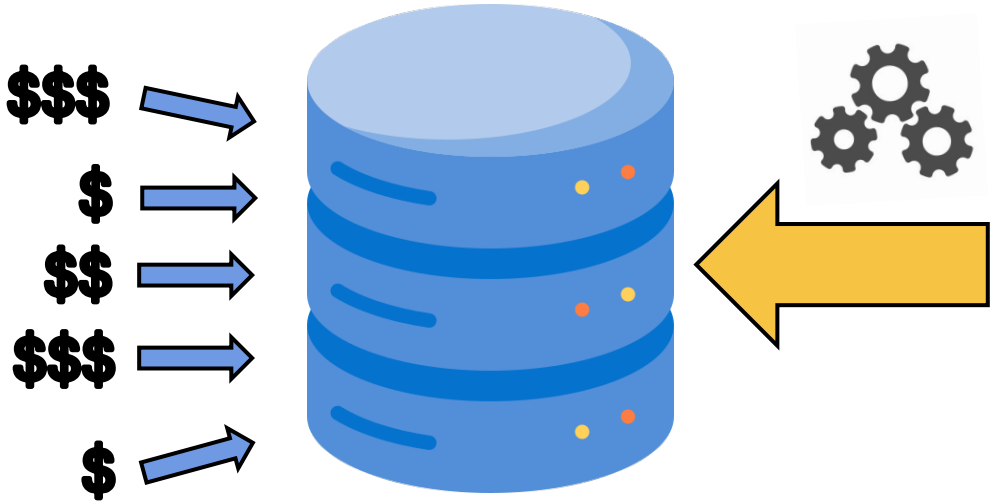
Linux page cache significantly affects performance

LRU-approximation algorithm – **doesn't work well for all workloads!**



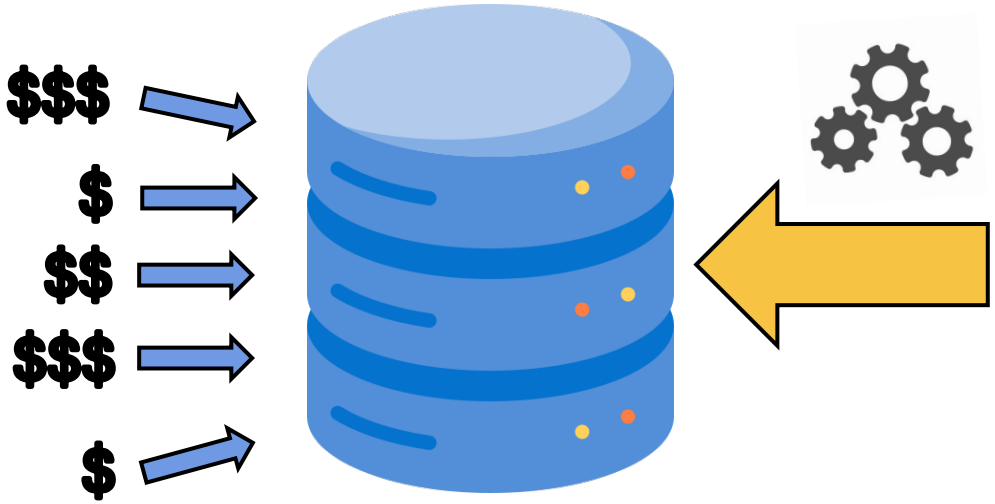
Linux page cache struggles with some workloads

- Many tiny transactions
- A few massive background tasks



Linux page cache struggles with some workloads

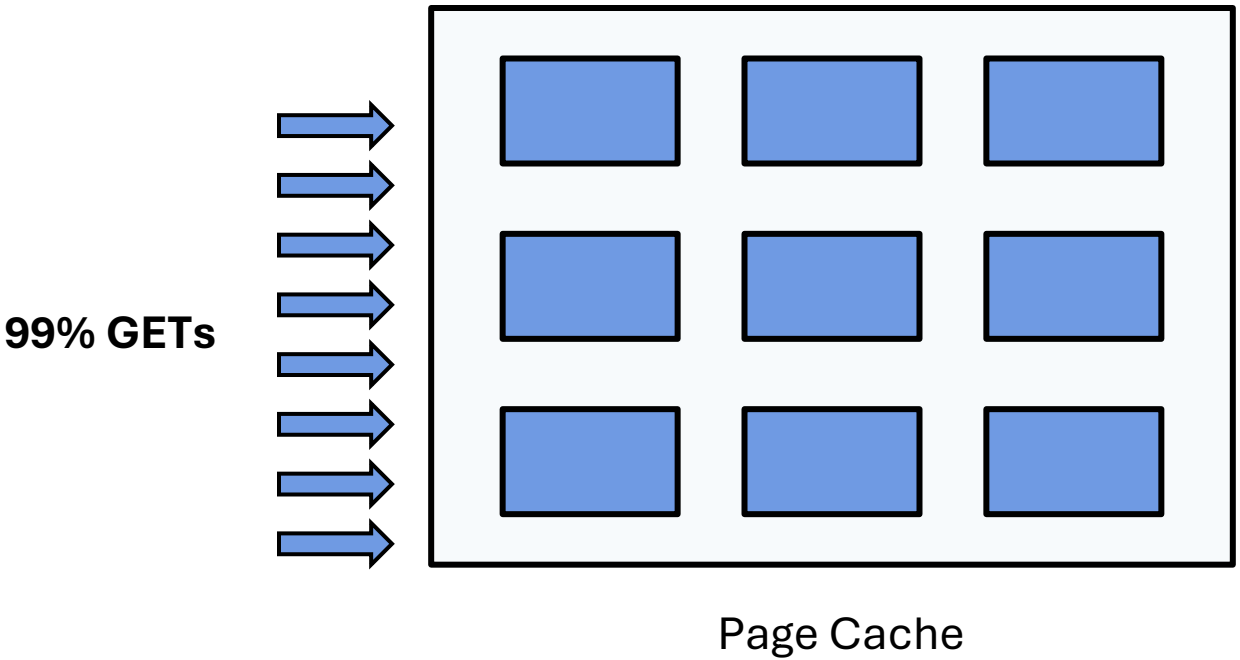
- Many tiny transactions
- A few massive background tasks



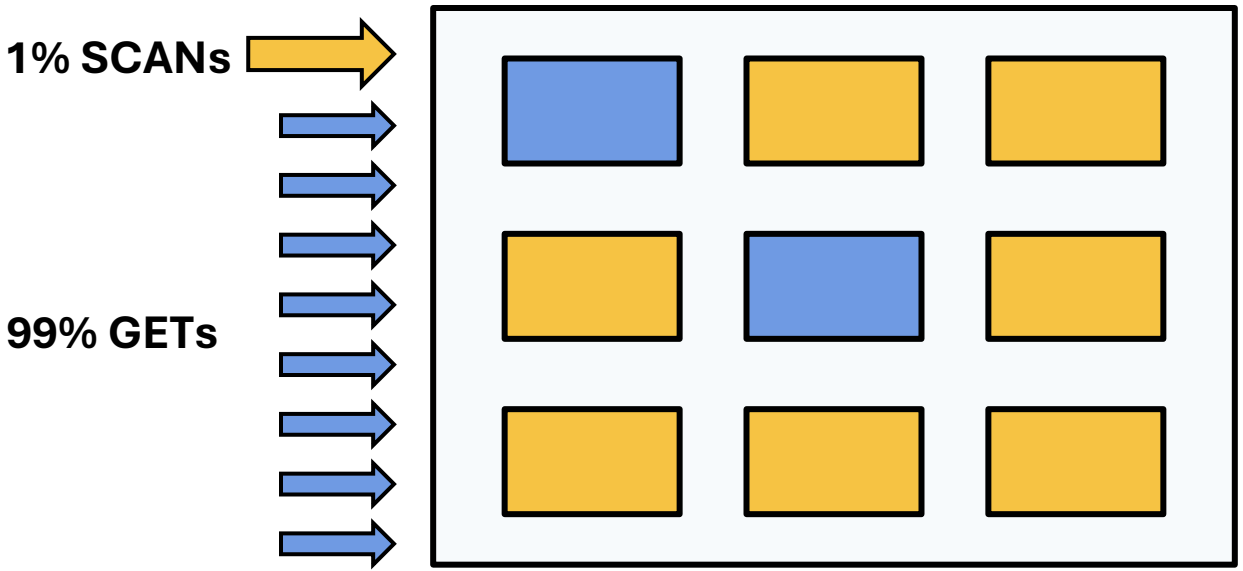
HTAP-like:
Mix of Transactional (GET) and Analytical (SCAN)

Shinjuku (NSDI ’19),
Syrup (SOSP ‘21)

Linux page cache struggles with some workloads

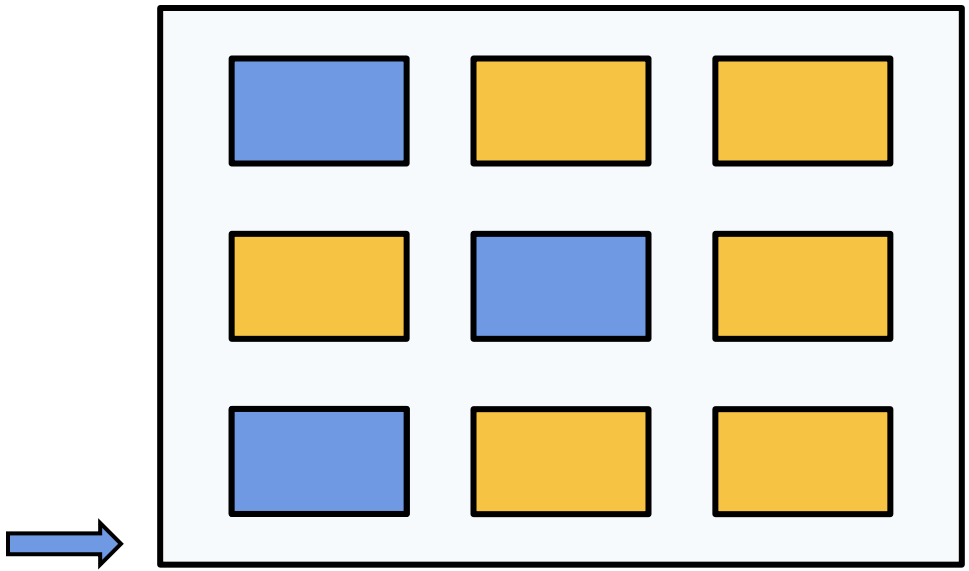


Linux page cache struggles with many workloads



Data is 99% SCAN and 1% GET

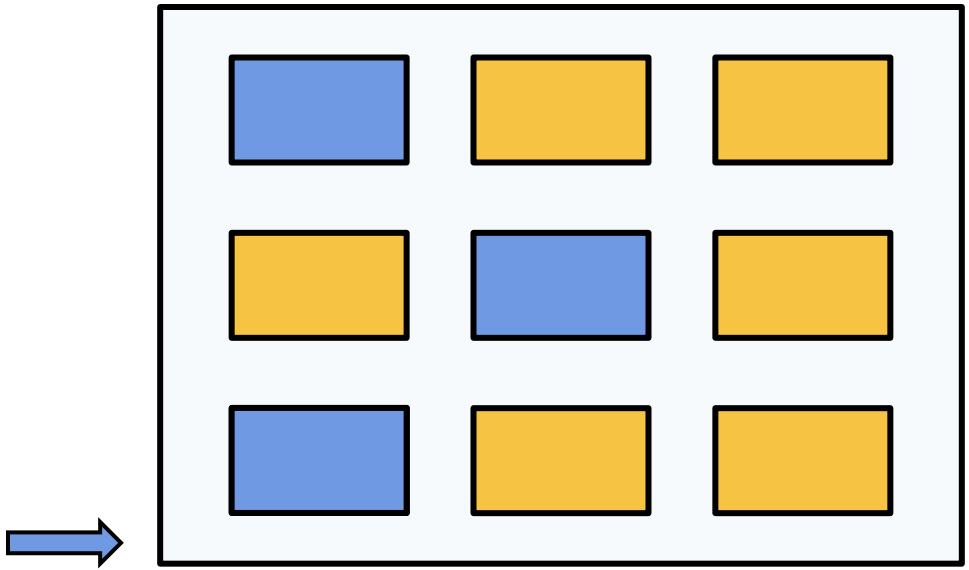
Linux page cache struggles with many workloads



Page Cache

Linux page cache struggles with many workloads

We've been thrashed!



Page Cache

The page cache is not flexible enough

We are leaving performance on the table

Existing solutions

- Change the policy
- Userspace hints
- Application-level caching

Changing the policy is hard

MGLRU merged in Linux v6.1 as new experimental policy

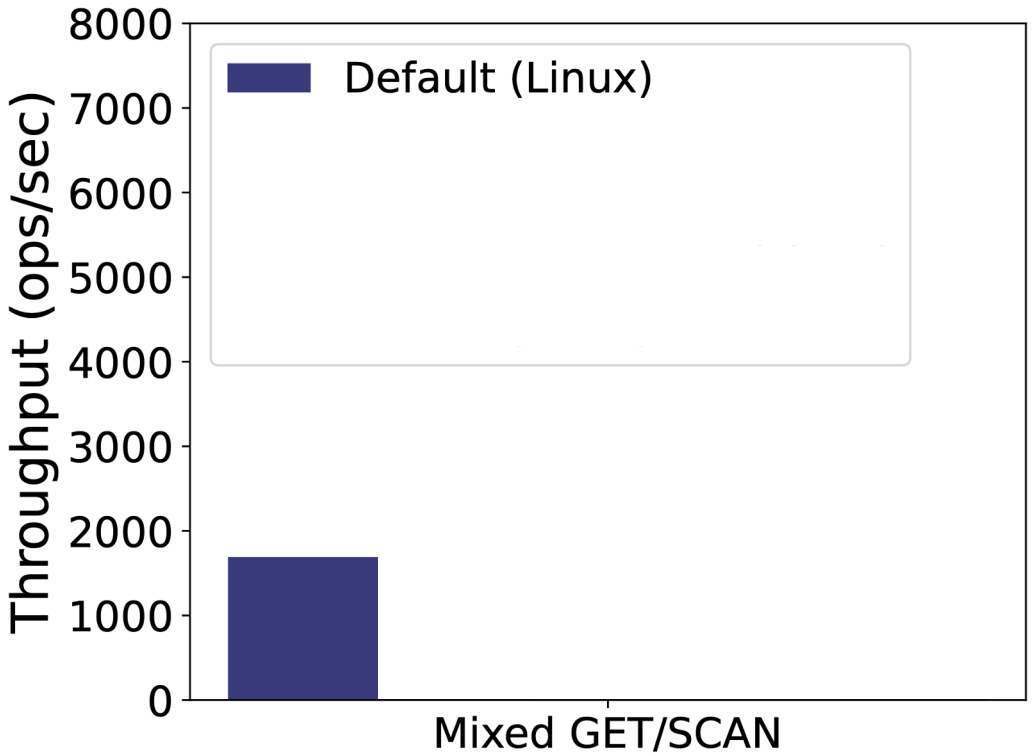
- Several years of effort
- Still not default upstream
- Global policy change

`fadvise()` isn't powerful enough

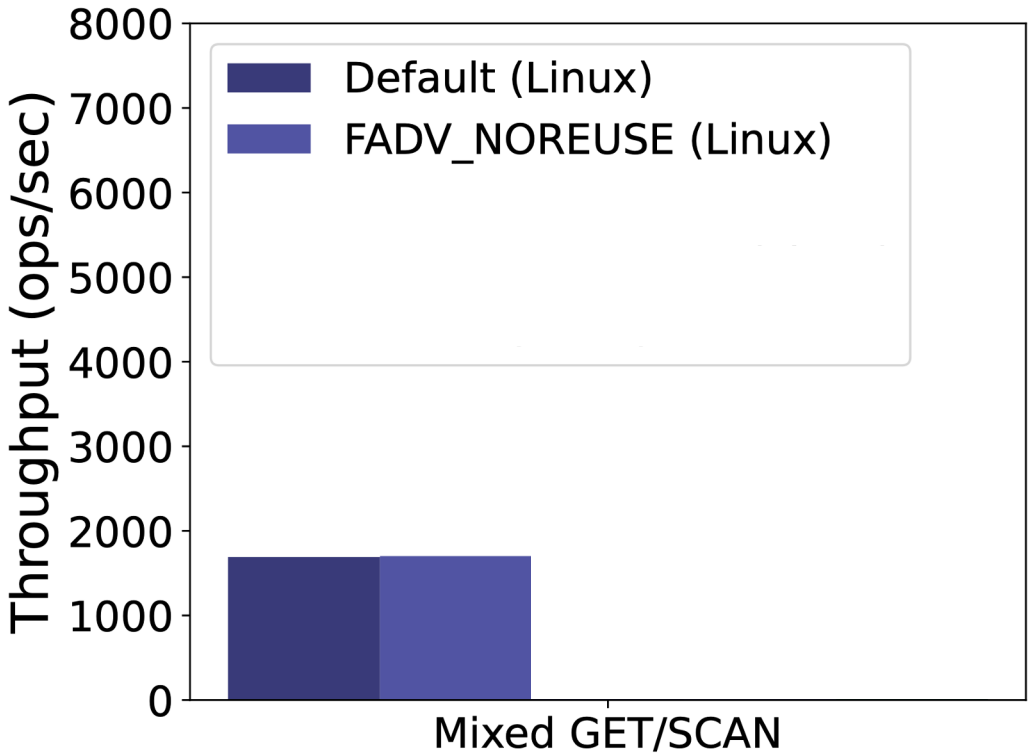
Added in v2.5.60

- 6 flags
- **`FADV_NOREUSE`** was a no-op until v6.3

Existing hint interfaces are insufficient

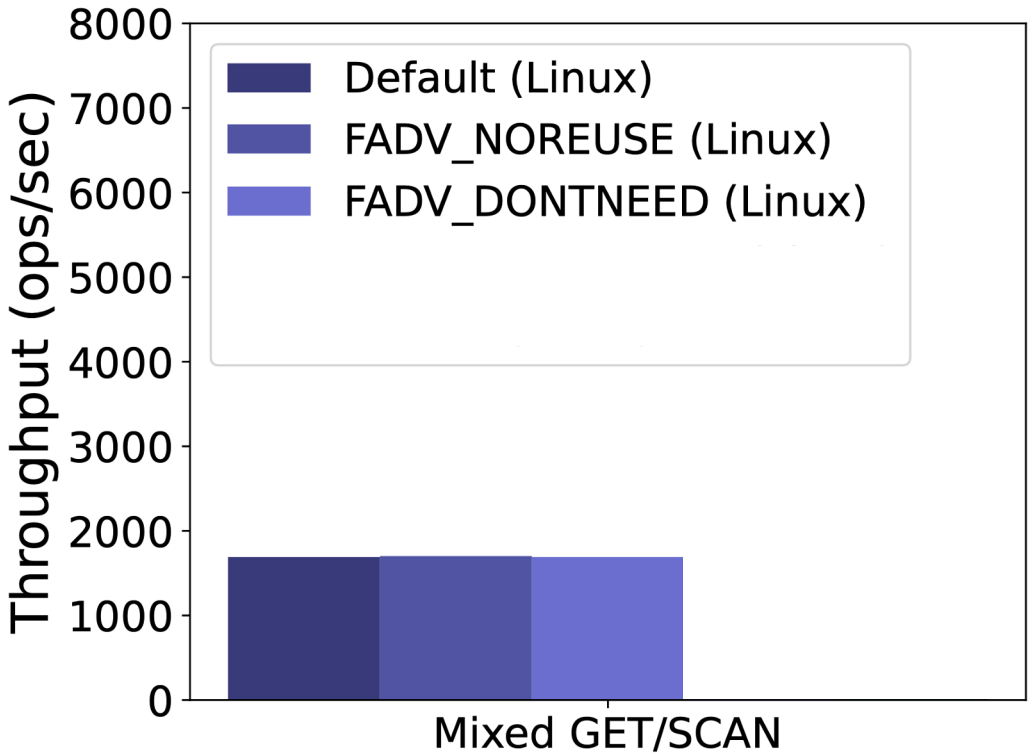


Existing hint interfaces are insufficient



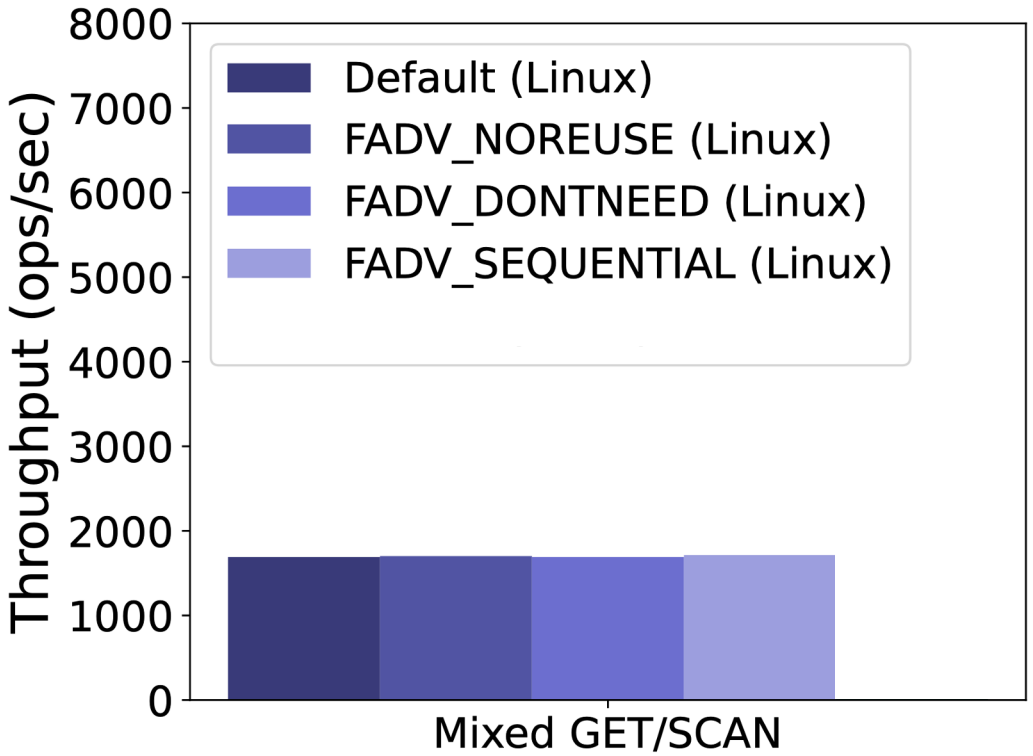


Existing hint interfaces are insufficient



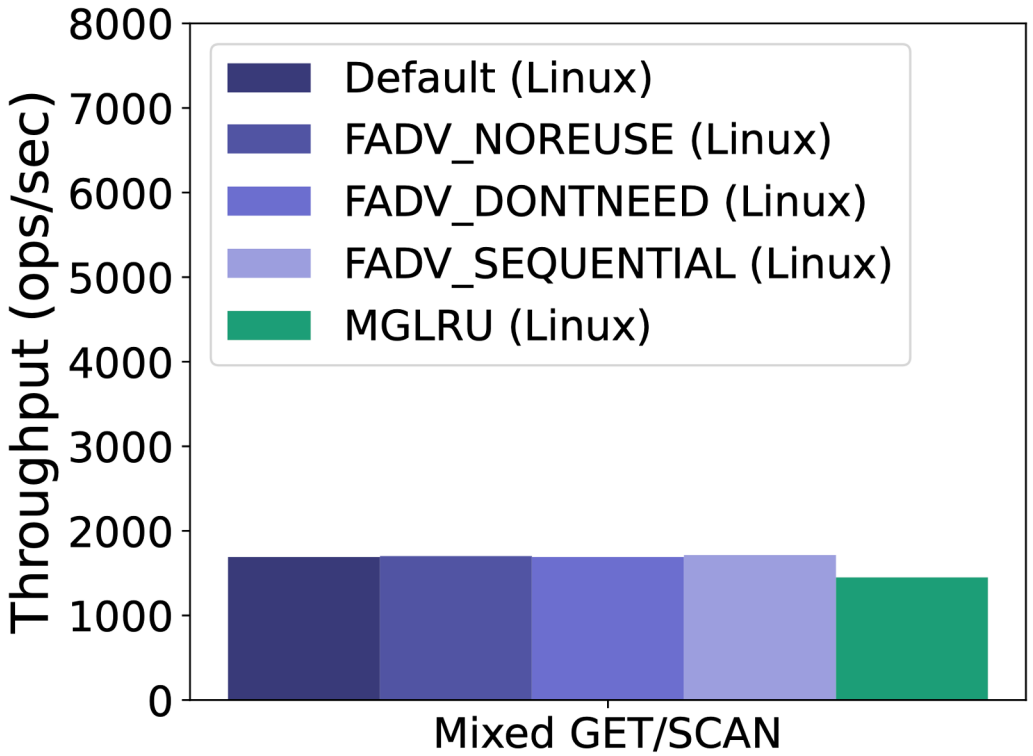


Existing hint interfaces are insufficient





Existing hint interfaces are insufficient





Hints don’t work!

How can we improve page cache behavior?

	Change the policy	Userspace hints	Application-level caching
Customizability	✅	❌	✅
Simplicity	❌	✅	❌
Isolation	❌	✅	✅ / ❌

We need to change the underlying policy

Like scheduling, networking, file systems, etc.

cache_ext: Custom page cache policies

- Simplicity: No kernel changes for developers ✅
- Isolation: Per-cgroup policies ✅
- Customizability:

cache_ext: Custom page cache policies

- Simplicity: No kernel changes for developers ✅
- Isolation: Per-cgroup policies ✅
- Customizability: ✅



eBPF to the rescue!

- Similar approaches taken for scheduling, networking, etc.

  sched_ext, Syrup (SOSP ‘21), ghOSt (SOSP ‘21), XDP, XRP (OSDI ‘22), …

cache_ext interface

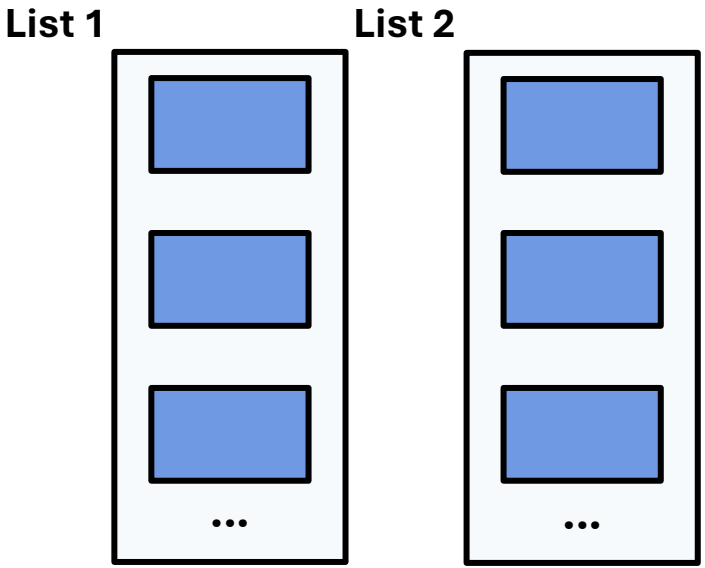
- struct_ops hooks added to page cache implementation
- ~200 LoC in page cache code (mostly batching eviction)

```
// Policy function hooks
struct cache_ext_ops {
    s32  (*policy_init)(struct mem_cgroup *memcg);
    // Propose folios to evict
    void (*evict_folios)(struct eviction_ctx *ctx,
         struct mem_cgroup *memcg);
    void (*folio_added)(struct folio *folio);
    void (*folio_accessed)(struct folio *folio);
    // Folio was removed: clean up metadata
    void (*folio_removed)(struct folio *folio);
    char name[CACHE_EXT_OPS_NAME_LEN];
};

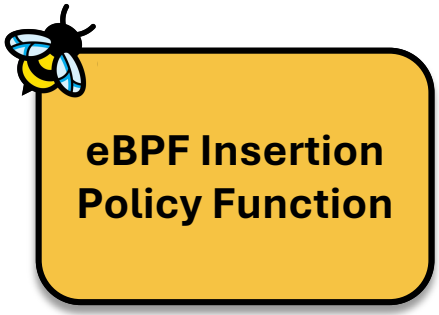
struct eviction_ctx {
    u64 nr_candidates_requested; /* Input  */
    u64 nr_candidates_proposed;  /* Output */
    struct folio *candidates[32];
};
```

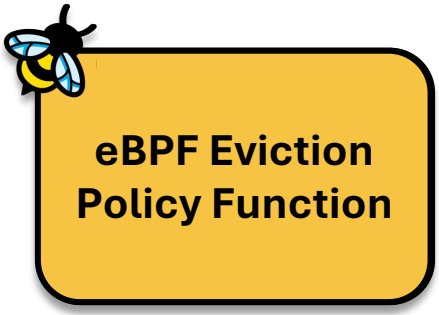
cache_ext: Simple API yields powerful policies

eBPF Eviction Lists



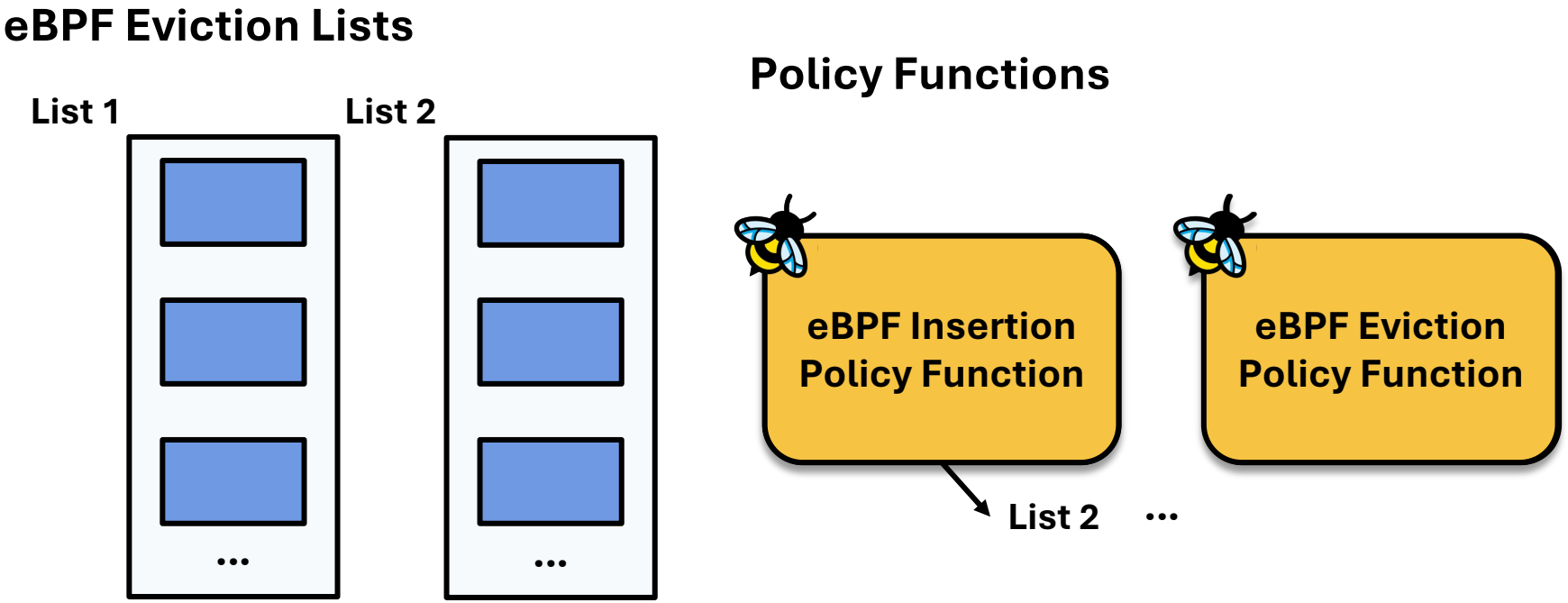
Policy Functions



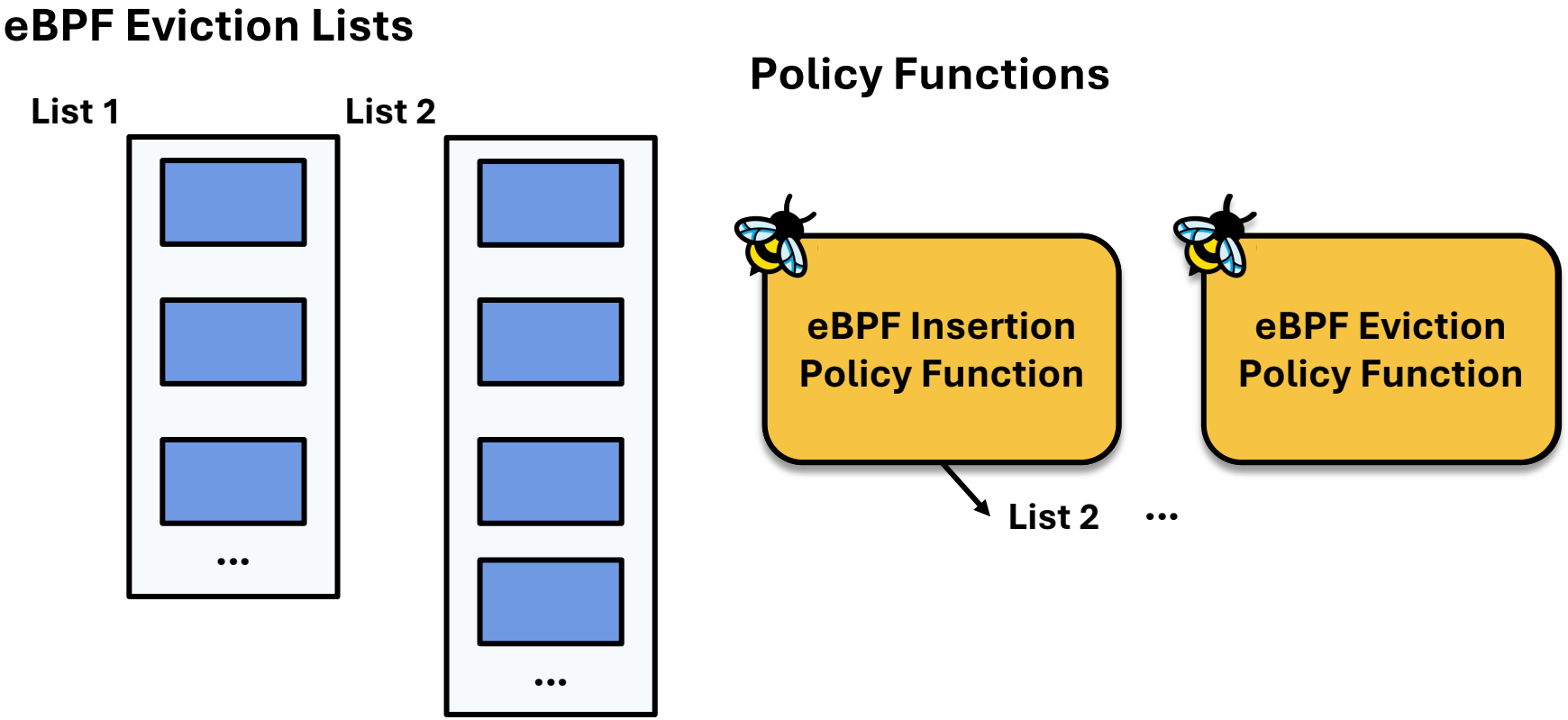


...

cache_ext: Simple API yields powerful policies

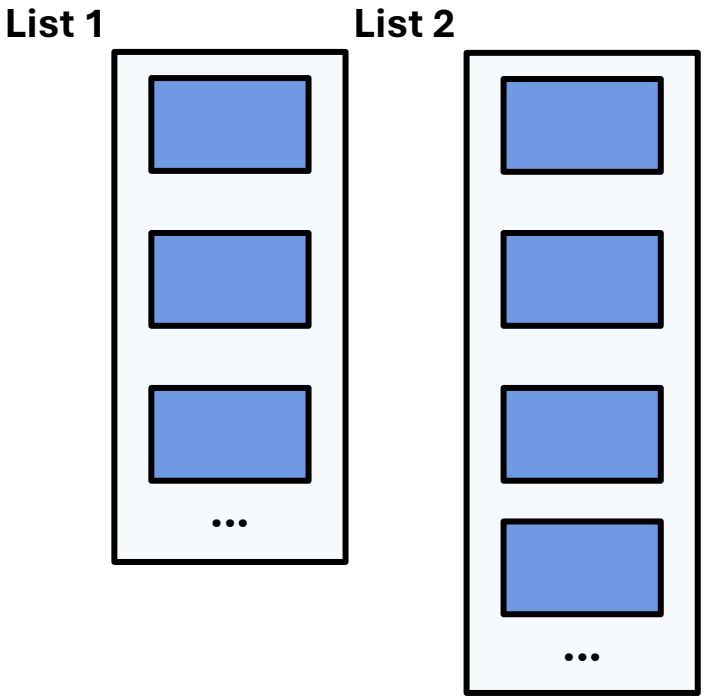


cache_ext: Simple API yields powerful policies

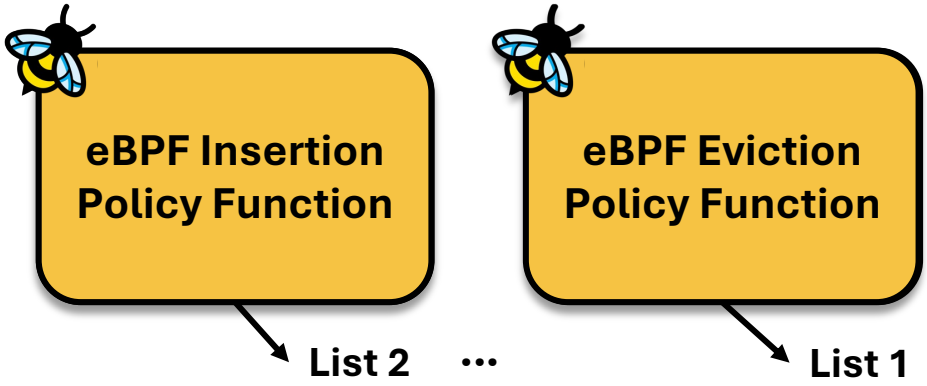


cache_ext: Simple API yields powerful policies

eBPF Eviction Lists

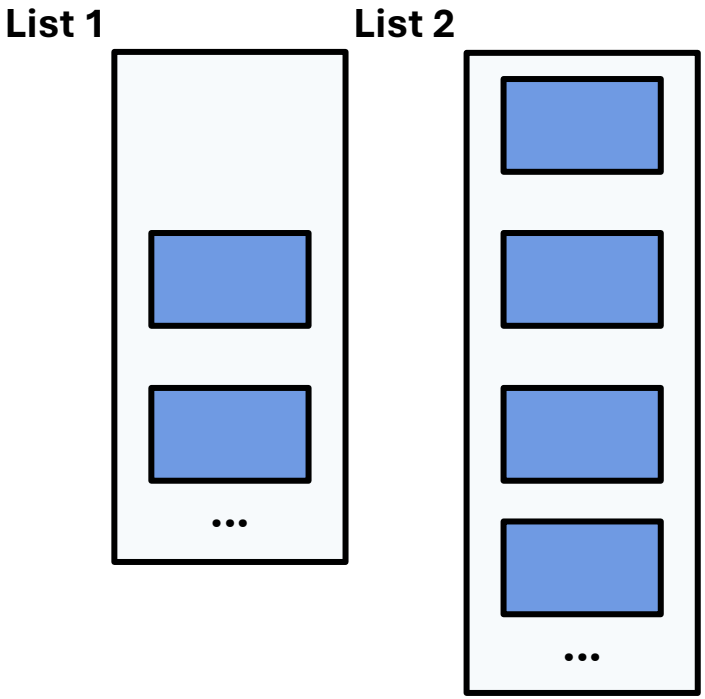


Policy Functions

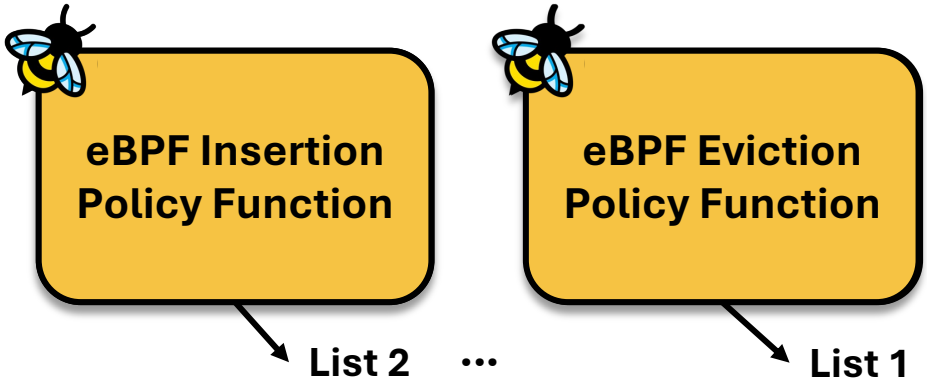


cache_ext: Simple API yields powerful policies

eBPF Eviction Lists

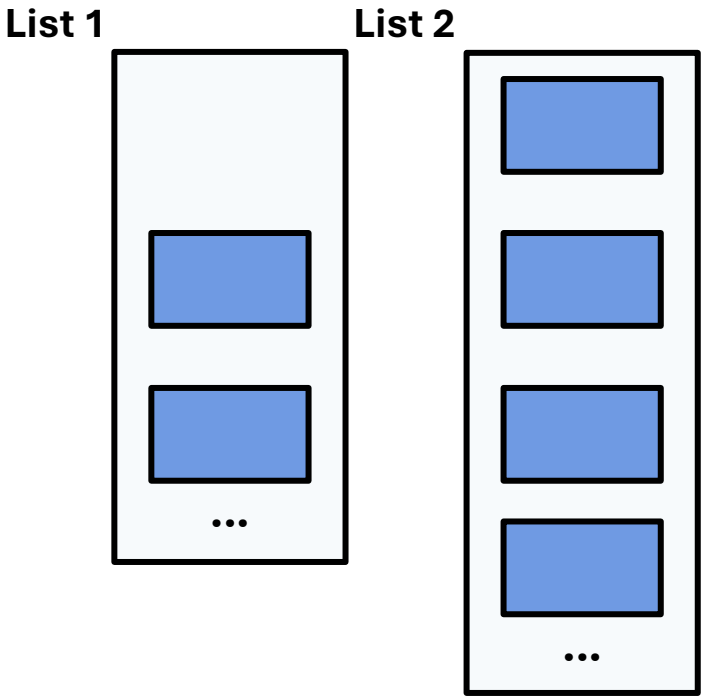


Policy Functions

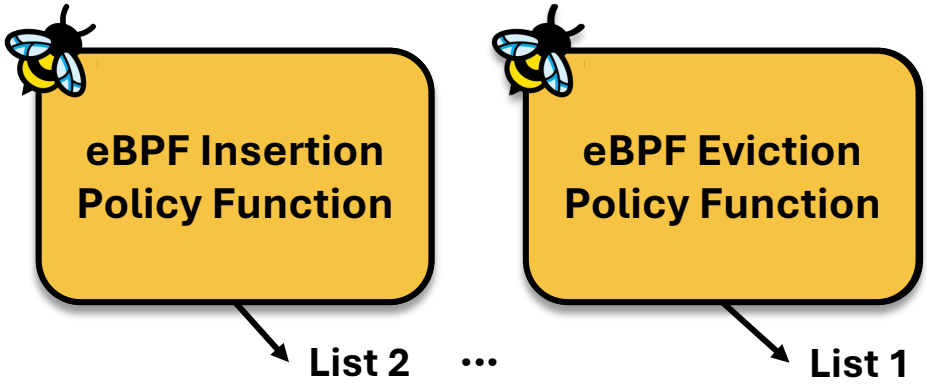


cache_ext: Simple API yields powerful policies

eBPF Eviction Lists



Policy Functions



Most policies can be implemented with lists!

cache_ext list kfuncs

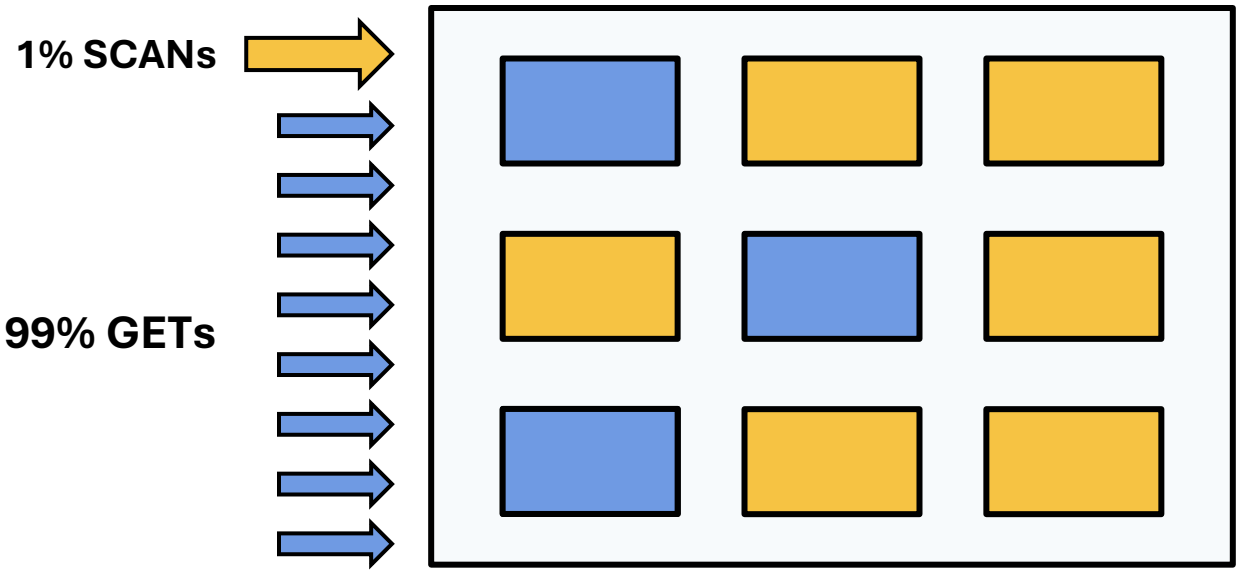
Eviction lists are similar to sched_ext DSQs

Eviction list API
`u64 list_create(struct mem_cgroup *memcg)`
`int list_add(u64 list, struct folio *f, bool tail)`
`int list_move(u64 list, struct folio *f, bool tail)`
`int list_del(struct folio *f)`

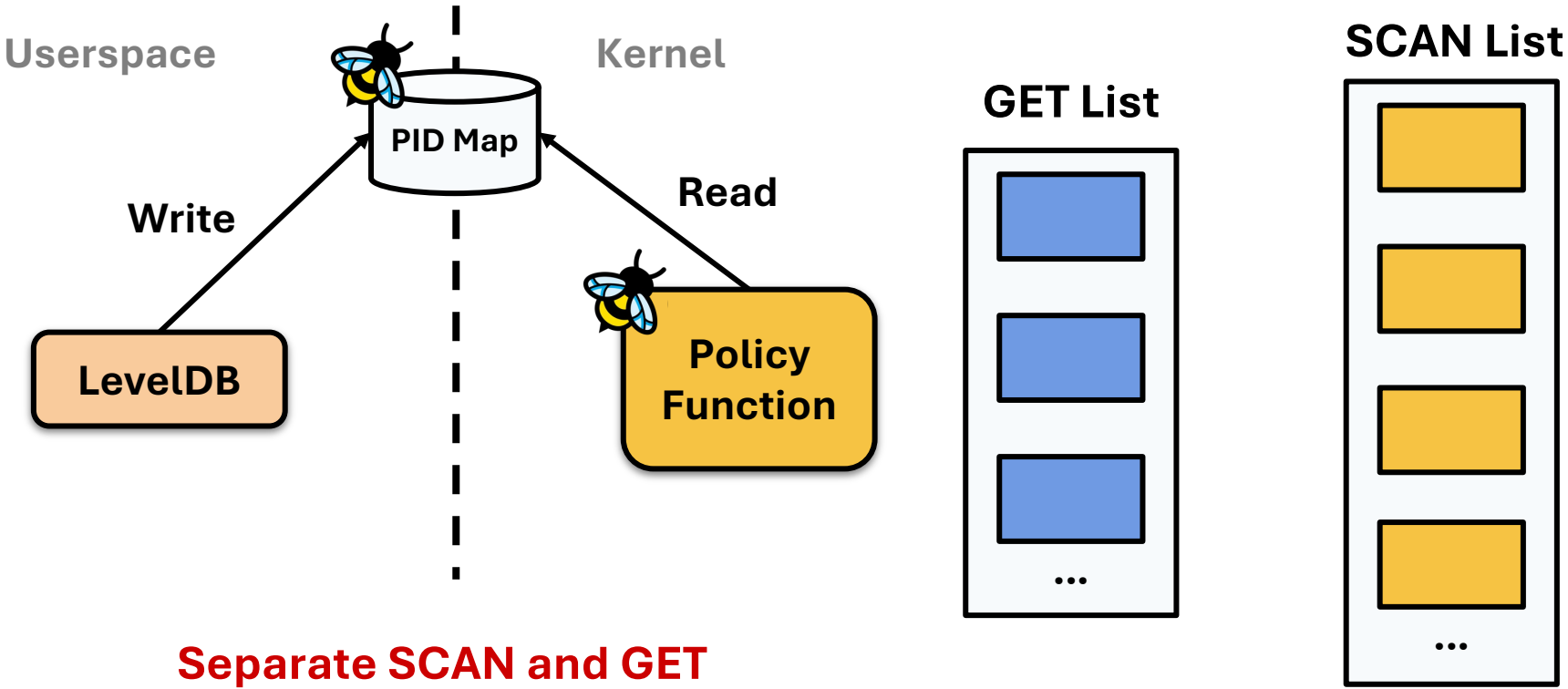
`int list_iterate(struct mem_cgroup *memcg, u64 list,`
`  s64(*iter_fn)(int id, struct folio *f),`
`  struct iter_opts *opts, struct eviction_ctx *ctx)`

Note: These kfuncs are namespaced with a cache_ext prefix

GET/SCAN doesn’t work well with default policy

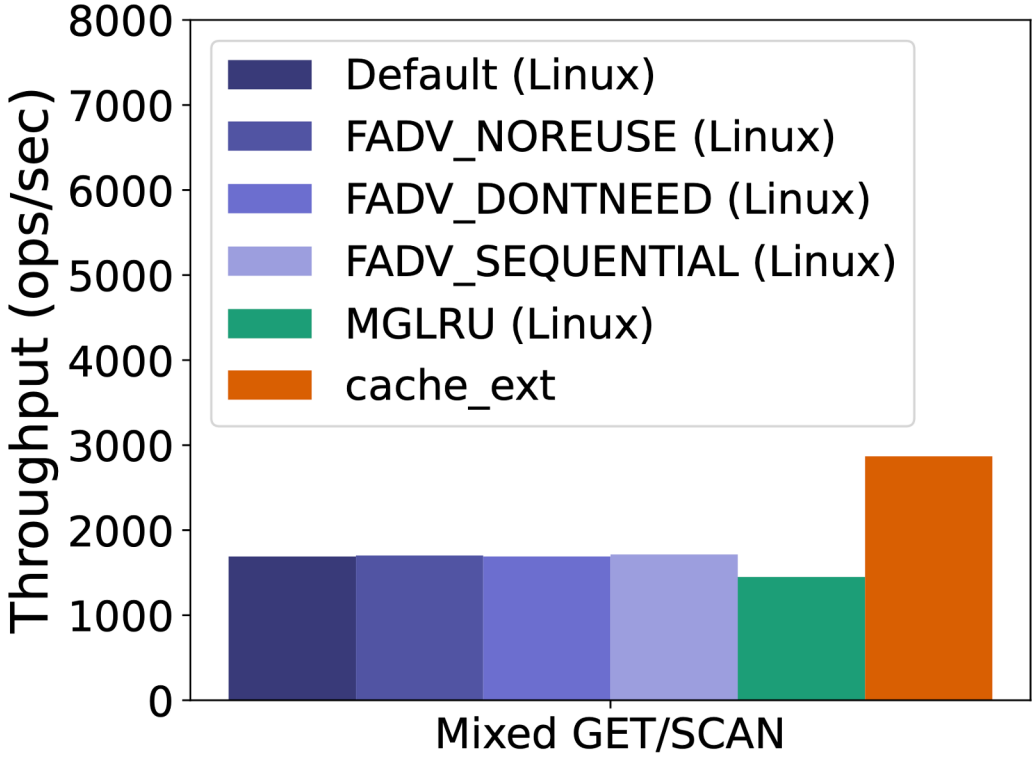


cache_ext enables application-informed policies



Separate SCAN and GET data by request PID

Application-informed policies improve performance





70% GET throughput increase

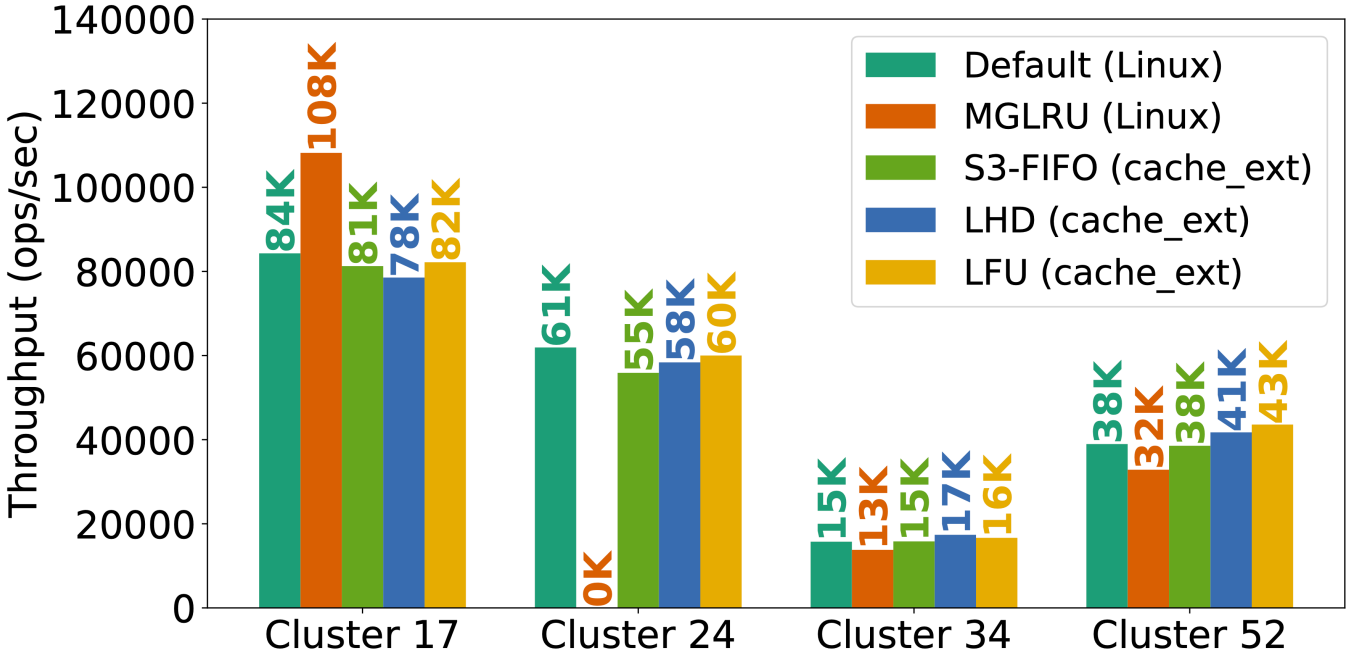
57% tail latency decrease

LFU policy

```
u64 lfu_list;
int lfu_policy_init(struct mem_cgroup *cg) {
    lfu_list = list_create(cg);
    return 0;
}
void lfu_folio_added(struct folio *folio) {
    u64 freq = 1;
    list_add(lfu_list, folio, true); // Add to tail
    bpf_map_update_elem(&freq_map, &folio, &freq);
}
void lfu_folio_accessed(struct folio *folio) {
    u64 *freq = bpf_map_lookup_elem(&freq_map, &folio);
    __sync_fetch_and_add(freq, 1); // Increment freq
}
```

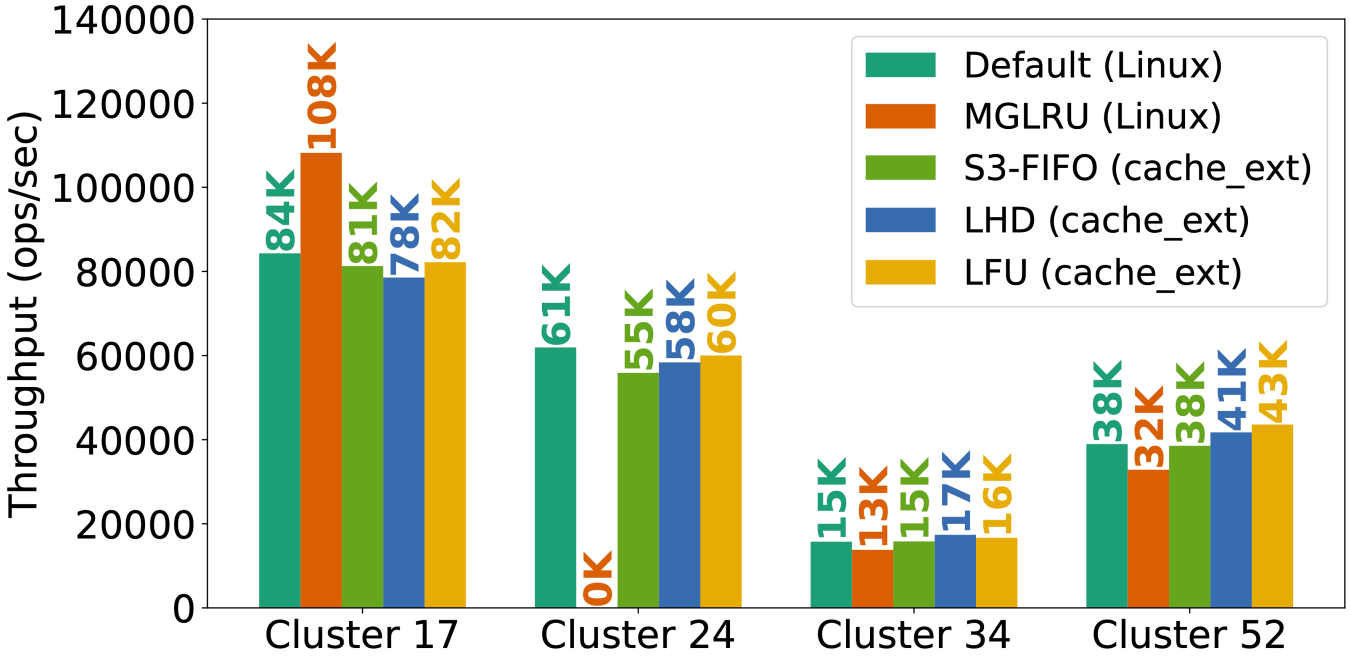
```
long score_lfu(int id, struct folio *folio) {
    return bpf_map_lookup_elem(&freq_map, &folio);
}
void lfu_evict_folios(struct eviction_ctx *ctx, struct
     mem_cgroup *cg) {
    struct iter_opts opts = { /* Set scoring mode */ };
    list_iterate(cg, lfu_list, score_lfu, &opts, ctx);
}
void lfu_folio_removed(struct folio *folio) {
    bpf_map_delete_elem(&freq_map, &folio);
}
```

Customizability yields better performance



Twitter traces (OSDI ‘20),
S3-FIFO (SOSP ‘23),
LHD (NSDI ‘18)

Customizability yields better performance



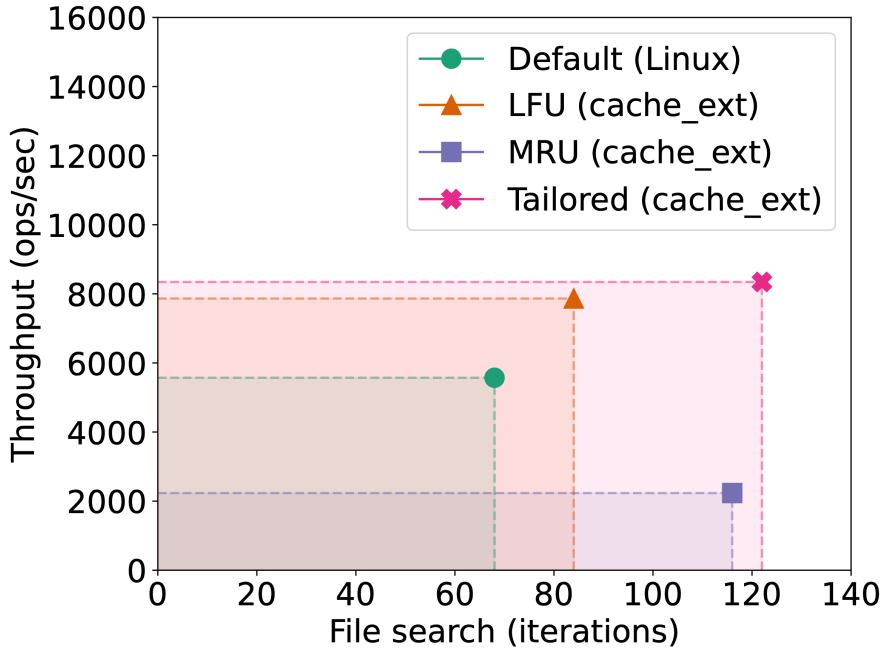
Twitter traces (OSDI '20), S3-FIFO (SOSP '23), LHD (NSDI '18)

Also ran with YCSB: Up to 37% throughput improvement

No one policy performs best in all cases!

Per-cgroup policies

cgroup 1



cgroup 2

Optimal performance when each cgroup runs a policy matching its workload:

50% and 80% improvements over baseline

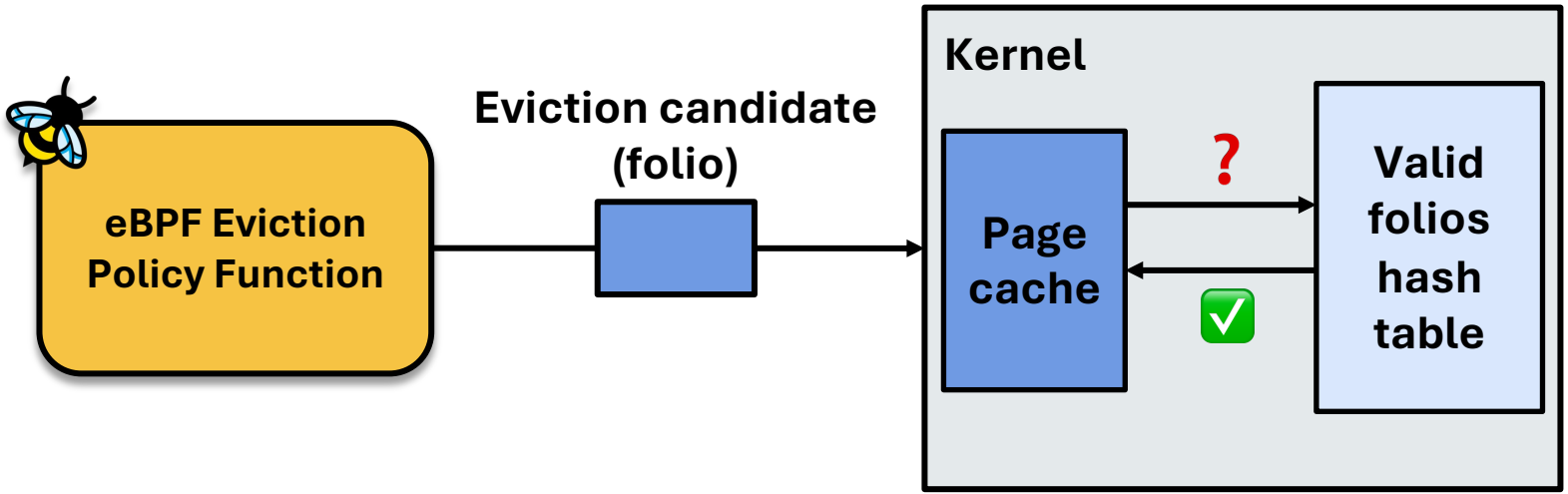
Per-cgroup struct_ops: Implementation

- New libbpf bpf_map_attach() API
  - Takes map and cgroup fds
- New BPF link type: contains a struct_ops map and a struct cgroup *
- struct cgroup_bpf contains cache_ext struct_ops data
  - Kind of a hack...

Also discussed on-list for sched_ext, BPF OOM?

Challenge: Ensuring memory safety

- eBPF eviction function can return any value… could be invalid
- Solution: Kernel maintains “valid folios” hash table
- Also stores the eviction list nodes



Memory safety: Better solution

- Reference count properly + use trusted pointers
- Why didn’t we do this initially?
- Folios are placed on two lists: kernel’s LRU list and cache_ext list
- This complicates reference counting
- Should be on one list: reuse folio->lru
- Can then drop the valid_folios hash table entirely

cache_ext enables modern page cache usage

- In order to maximize performance, must be able to experiment
- Policies can be reused across applications
- Applications can have their own per-cgroup policies (multi-tenancy)

More Questions

- Is MM open to BPF and/or custom page cache policies?
- Interest from community?
  - Lots of enthusiasm from academia, what about industry?
- MGLRU BPF extensions?
  - These were discussed at LSFMM 2022 and LPC 2024, but don’t seem to exist (publicly)

Thank you!

- cache_ext enables custom page cache policies to better match applications
- Up to 70% higher throughput and 58% lower tail latency
- More in the paper: Policies, multi-tenancy, admission filter, security, overhead…

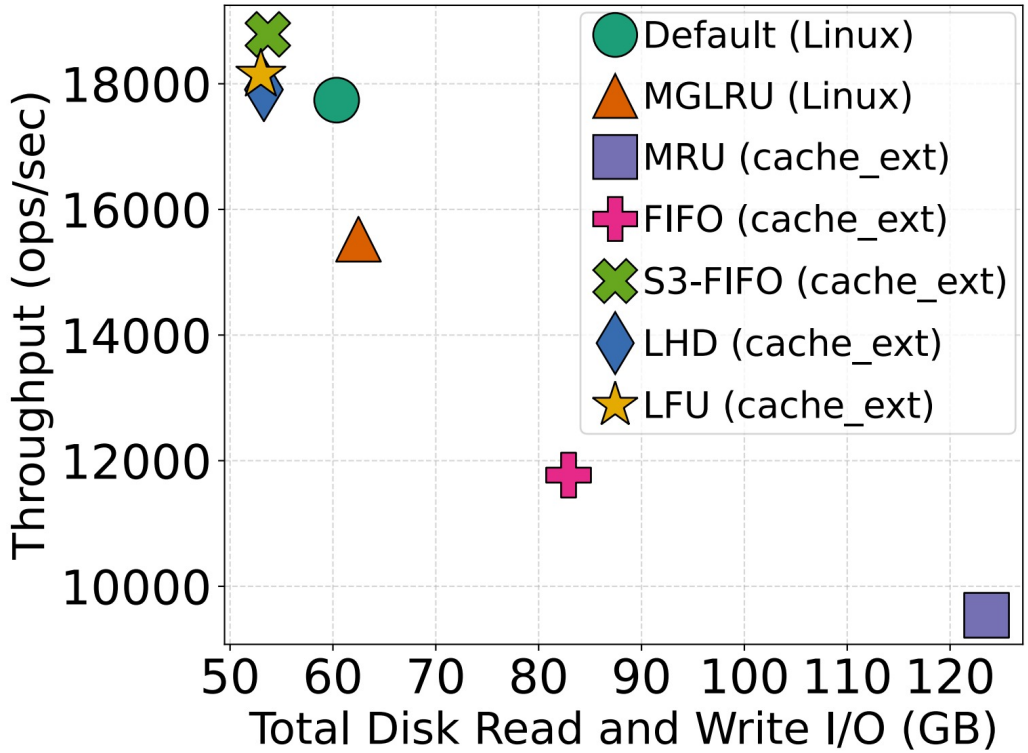
Paper and code:



Tal Zussman
tz2294@columbia.edu

github.com/cache-ext/cache_ext

Disk access reduction



Policy complexity

Policy	eBPF LoC	Userspace LoC
Admission filter	35	262
FIFO	56	131
MRU	101	101
LFU	215	110
S3-FIFO	287	157
GET-SCAN	324	112
LHD	367	165
MGLRU	689	105